A Relational Database for Brazilian Jiu-Jitsu Match Tracking

Student number: 3520872

***Abstract*—This report documents the design and implementation of a relational database that tracks Brazilian Jiu-Jitsu fighters, the gyms they train at, and the results of their matches. The database was designed using five entities and normalised to remove redundancy, then built in MySQL using phpMyAdmin and filled with sample data that was imported from CSV files. Six SQL queries were then written to test the database, and one part of the scenario that would be difficult to implement in SQL is discussed.**

I. Introduction

Brazilian Jiu-Jitsu(BJJ) is a grappling martial art which consists of 1 vs 1 matches between fighters, in typical competitions fighters can win their matches via submissions, points, and various other methods. This means tracking fighter's gyms and results in a single spreadsheet can quickly lead to repeated data and inconsistent records. Therefore, the aim of this project is to tackle this problem by managing scenarios as a relational database that can answer questions such as who won a match, their method of victory, and how often each fighter wins.

The database consists of five entities: gym, fighter, VictoryType, match, FighterMatchPerformance. This is because a fighter takes part in many matches which involve two fighters, FighterMatchPerformance resolves this many-to-many relationship by using a composite primary key that is made of both FighterID and MatchID. Win rate is a derived value that is calculated by a query rather than being stored, this avoids redundancy and out of date values. The database design assumes that each match has two performance records, meaning that each fighter belongs to one gym, each match has one victory method, and all data are fictional. Section II will present the design, Section III the implementation, Section IV the queries, and Section V will discuss the complexity.

II. Database Design

A. Entities and Relationships

The database consists of five entities: Gym, Fighter, VictoryType, Match, and FighterMatchPerformance. A Gym can have zero or more Fighters, but every Fighter must belong to exactly one Gym, so GymID is the foreign key in the Fighter table. A VictoryType can apply to zero or more Matches, but every Match must have one VictoryType, so TypeID is the foreign key in the Match table. A Fighter can appear in zero or more FighterMatchPerformance records, and a Match must have at least one FighterMatchPerformance record, which resolves the many-to-many relationship between Fighter and Match. FighterMatchPerformance uses a composite primary key made of FighterID and MatchID and stores PointsScored and Result. The diagram uses crow's foot notation, where a circle means optional (zero allowed) and a bar means mandatory (at least one required).

B. Normalisation

Each entity was assessed against first, second, and third normal form. All five entities satisfy 3NF, as shown in Table I.

TABLE I
Normal form of each entity

Entity	NF	Justification
Gym	3NF	Name and Location depend only on GymID, so there are no partial or transitive dependencies.
Fighter	3NF	All attributes depend on FighterID, and the gym details are stored in Gym so they are not repeated for each fighter.
VictoryType	3NF	MethodName depends only on TypeID, so each method is stored once and matches cannot hold different spellings of it.
Match	3NF	EventDate and TypeID depend only on MatchID, and the winner is not stored here as the result is held in FighterMatchPerformance.
FighterMatch-Performance	3NF	PointsScored and Result depend on the whole composite key (a fighter in a match), not on FighterID or MatchID alone, so there is no partial dependency.

C. Data Types and Integrity

Each entity has a primary key, each foreign key references an existing record, and invalid values are rejected. GymID, FighterID, TypeID, and MatchID are integers and are primary keys. GymName and FighterName are VARCHAR and NOT NULL, while GymLocation is VARCHAR and can be NULL. TypeName is VARCHAR and NOT NULL UNIQUE, and MatchDate is DATE and NOT NULL.

TypeID in Match is an integer foreign key to VictoryType and NOT NULL, so every Match has one VictoryType. GymID in Fighter is an integer foreign key to Gym and NOT NULL, so every Fighter belongs to exactly one Gym.

FighterMatchPerformance has FighterID and MatchID as integer foreign keys to Fighter and Match, both NOT NULL, and the composite primary key (FighterID, MatchID) prevents the same Fighter from being recorded twice in the same Match. PointsScored is an integer with DEFAULT 0 and CHECK (PointsScored ¿= 0). Result is VARCHAR with CHECK (Result IN ('Win', 'Loss')).

The foreign keys enforce referential integrity, the primary keys enforce entity integrity, and the CHECK constraints enforce domain integrity. The rule that a Match must have at least one FighterMatch-Performance record is enforced by the application or a trigger because a foreign key from Match to FighterMatchPerformance would create a circular reference. Win rate is calculated by a query from Fighter-MatchPerformance rather than stored, which avoids redundancy and out-of-date values.

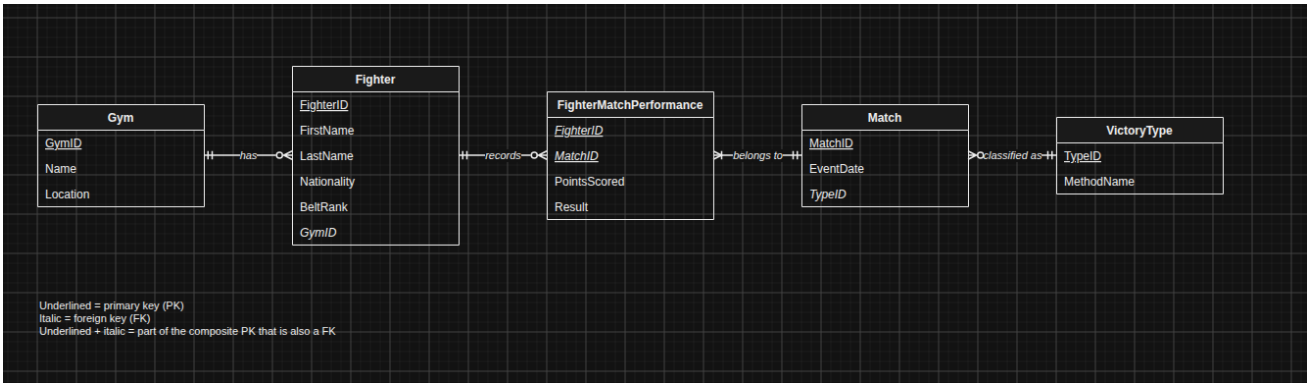


Fig. 1. Entity-relationship diagram. Underlined attributes are primary keys, italic attributes are foreign keys, and underlined italic attributes form the composite primary key of FighterMatchPerformance.

III. Implementation

The file create_tables.sql creates the tables. Gym and VictoryType are created first because Fighter and Match have foreign keys pointing to them. FighterMatchPerformance is created last because it references both Fighter and Match. Each table uses the InnoDB engine and defines its primary key and foreign key constraints.

Five CSV files, one per entity, were imported using LOAD DATA LOCAL INFILE in load_data.sql. The files had no header row and used comma separators. Parent tables were loaded before child tables for the same foreign key reason as the creation order.

One INSERT INTO statement was written for each entity to demonstrate manual insertion. These statements are also in load_data.sql below the LOAD DATA block.

A SELECT COUNT(*) union query at the end of load_data.sql confirmed all rows loaded correctly. Screenshots of the results were taken in phpMyAdmin as evidence.

IV. Queries

Six queries were written to test different SQL features.

- Query 1 uses `LIKE 'Mc%'` to return fighters whose surname begins with Mc.
- Query 2 uses a four-table `JOIN` to return each win showing fighter name, match date and victory method.
- Query 3 uses `PointsScored > 5` to return performances where a fighter scored more than five points.
- Query 4 uses `GROUP BY` with `COUNT` and `SUM` to calculate total fights and win percentage per fighter.
- Query 5 uses `IN ('Blue','Purple')` to return fighters holding a blue or purple belt.
- Query 6 uses `UPDATE` to promote three fighters to Blue belt, with a `SELECT` before and after to confirm the change.

V. Complexity Discussion

The schema allows any number of FighterMatchPerformance rows for a given match. A real BJJ match always has exactly two fighters, so ideally the database would enforce a count of two. A standard foreign key can only enforce that a referenced row exists; it cannot enforce a count.

SQL alone cannot fix this. A CHECK constraint cannot reference another table's row count. The only SQL solution is a trigger that counts existing rows for a given MatchID before each insert and raises an error if the count would exceed two. Triggers add complexity and are harder to test, and they are not covered by basic DDL.

An alternative is to store Fighter1ID and Fighter2ID directly on the Match table. That would guarantee exactly two fighters, but it introduces a partial dependency and breaks 3NF. The current design accepts the limitation and leaves enforcement to the application layer, which keeps the schema normalised.